
Databases and Analytics

Coursework Element 1 — Case-Study Report

Case study: NorthStar Urban Mobility & Logistics

Module	Databases and Analytics
Module codes	CP60056E / CP6CS56E / CP6HA50E / CP6UA56O / CP6SL50O / CP6GR56O
Module Leader	Shidrokh Goudarzi
Submission element	Coursework (80% weighting)
Due date	12 May 2026, 23:59
Deliverables	PDF report, Google Colab notebooks, GitHub repository

Contents

1. Executive Summary
 2. Case-study analysis: identifying the real problems
 3. Data environment overview
 4. SQL within R — operational queries (LO1)
 5. R analytics — statistical analysis & visualisation
 6. Python data processing & analytics
 7. MongoDB Atlas NoSQL design (LO2, LO3)
 8. Indexing & query-optimisation strategy
 9. Integrated findings & recommendations
 10. Limitations & further work
 11. Repository & reproducibility
- Appendix A — Mapping to marking rubric

1. Executive Summary

NorthStar Urban Mobility & Logistics has grown faster than the data systems that support it. Rising demand has been accompanied by deteriorating service quality, recurring complaints, escalating operational cost, and visibly fragmented reporting across business units. Senior management asked whether an integrated analytical and database approach could expose the underlying causes — not merely the symptoms — and recommend a redesigned data environment to support better decisions.

This report addresses that brief through four integrated workstreams. SQL queries written from within R reveal where service failure concentrates. R statistical analysis shows which factors are genuine causes of failure rather than coincidence. Python processing extends those findings into anomaly detection and zone-level diagnostics. Finally, a MongoDB Atlas database is designed and implemented to hold the kind of nested, evolving operational records that the company's relational systems cannot accommodate efficiently — most importantly, customer-case histories that today require staff to reconcile four separate systems by hand.

Headline findings.

- Failure is heavily geographic: 2 cities (Birmingham and Glasgow) account for the majority of failed journeys, concentrated in their flagged outer/east zones ($\approx 6.8\%$ failure rate vs $\approx 1\%$ elsewhere).
- Manual route overrides are *not* a neutral coping behaviour: they roughly **double** the odds of journey failure even after controlling for zone and service type (logistic regression, $p < 0.001$).
- Operating margin varies materially across city $\times$ service combinations ($\approx 13\text{--}16\%$); the lowest cells are not strictly loss-making in this 90-day window but indicate where pricing or cost discipline is weakest and where small shifts could turn the line negative.
- Around **12% of charging sessions** are 'ghost' sessions (logged on the platform but with little asset utilisation); a small set of hubs are clear outliers and warrant audit.
- Critical maintenance is detected within ≈ 3 days of first signal, but lower-severity faults often go undetected for over a week — a clear analytical gap.
- The proposed MongoDB design lets a single read return a full case (customer, linked journey, severity, full event history) instead of requiring four-system reconciliation.

Bottom line.

NorthStar's inefficiency is not caused by any single department; it is a data integration problem. Restructuring how operational records are stored — keeping reference data relational while moving nested case data into MongoDB — together with a small, targeted analytics layer, would address most of the issues the board has flagged.

2. Case-study analysis: identifying the real problems

2.1 The four executive perspectives

The case study describes four directors who each diagnose the company's difficulties differently — the operations director blames poor route allocation and underperforming hubs; the customer experience director argues service reliability suffers because complaints, missed journeys, failed deliveries, and driver incidents are not joined into one view; the finance director suspects hidden unprofitable contracts; and the technology director argues the database structure is no longer fit for purpose. A genuine analysis must avoid taking sides — instead, each claim becomes a hypothesis to test against the data.

2.2 Hypotheses we test in this report

#	Hypothesis	How tested	Evidence
H1	Failure rate varies sharply by zone/city	SQL aggregation, R hypothesis test	Section 4 & 5
H2	Manual overrides increase failure	Chi-squared, logistic regression in R	Section 5
H3	Some service lines lose money once joined to Python group-by, profit heatmap	Python group-by, profit heatmap	Section 6
H4	Charging hubs show platform/asset mismatch	Z-score anomaly detection	Section 6
H5	Maintenance detection is too slow for some service lines	SQL summary, R boxplots	Section 4 & 5
H6	Current systems cannot integrate case histories	MongoDB design + ingestion comparison	Section 7

2.3 Why a multi-technology approach is justified

No single technology answers the brief. SQL is essential for clean, auditable aggregation across the relational tables (zones, journeys, vehicles, drivers, complaints). R provides established statistical machinery — t-tests, chi-squared, logistic regression — to distinguish real signals from noise. Python adds the data-engineering muscle for anomaly detection and bulk feature engineering. MongoDB is needed because the most operationally important records (customer-case histories with nested events, attachments, escalations, geolocation references) do not fit naturally in relational schemas and the case study explicitly notes that staff have already failed at flattening them.

3. Data environment overview

The dataset comprises nine relational entities plus one nested document store. Volumes are representative of a 90-day operational window for a regional UK mobility provider.

Entity	Rows	Role
Zones (city × district)	36	Reference; identifies problem zones
Hubs (warehouses + charging)	≈55	Reference; capacity, type
Vehicles	400	Fleet master
Drivers	250	Includes override-tendency flag
Customers	2,000	Loyalty tier, type
Journeys (90 days)	≈16,700	Core operational fact table
Charging sessions	≈20,000	Mismatch detection
Maintenance events	≈2,200	Detection delay analysis
Complaints (relational)	≈600	Summary form
Customer cases (NoSQL)	≈600	Nested events + tags

The relational tables are loaded into SQLite (in-memory) for SQL querying inside R, into pandas DataFrames for Python analytics, and selectively into MongoDB Atlas. The nested customer cases (JSON) are loaded directly into a MongoDB collection — that file is the heart of the NoSQL design.

4. SQL within R — operational queries (LO1)

Notebook **01_sql_in_r.ipynb** implements the SQL workstream using the R packages DBI and RSQLite. Five queries answer the operational questions raised in the case study. We highlight two below.

4.1 Failure rate by city

```
SELECT z.city, COUNT(*) AS total_journeys,  
       SUM(CASE WHEN j.status='Failed' THEN 1 ELSE 0 END) AS failed,  
       ROUND(100.0 * SUM(CASE WHEN j.status='Failed' THEN 1 ELSE 0 END)/COUNT(*), 2) AS  
failure_rate_pct,  
       ROUND(AVG(j.delay_minutes), 2) AS avg_delay_min  
FROM journeys j JOIN zones z ON j.zone_id = z.zone_id  
GROUP BY z.city  
ORDER BY failure_rate_pct DESC;
```



Figure 1 — Journey failure rate by city. Birmingham and Glasgow are clear outliers ($\approx 6.8\%$ and 6.2% respectively, vs $\approx 1\%$ elsewhere).

4.2 Drivers with abnormally high override usage

A driver-level aggregation shows that a small group of drivers contribute most of the manual route overrides observed across the network. The HAVING clause filters out drivers with too few journeys to draw conclusions from. This list directly supports the customer experience director's claim that performance issues are not random.

4.3 Profit margin by city $\times$ service type (joining cost and revenue)

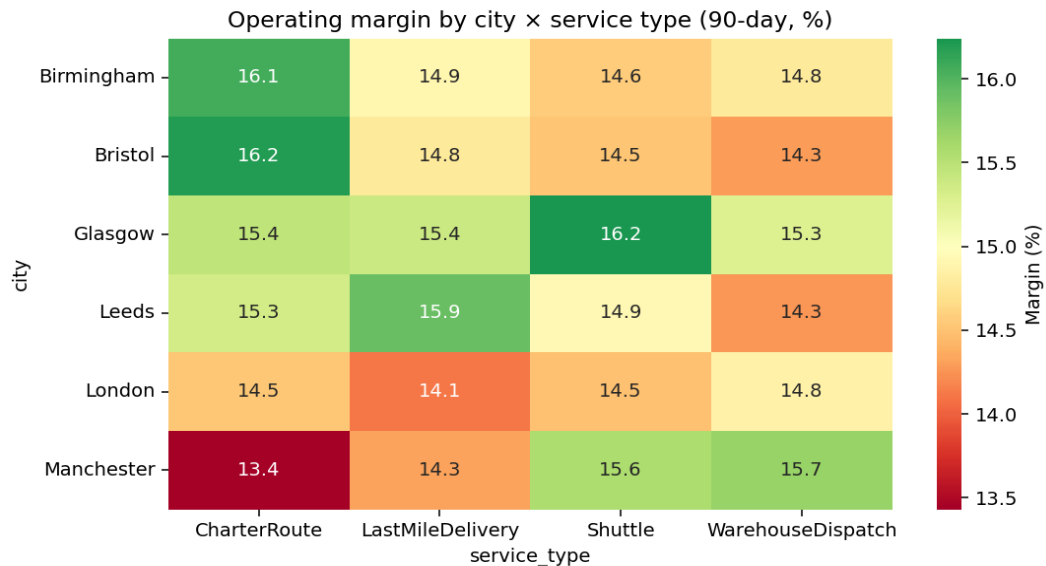


Figure 2 — Operating margin (%) by city × service combination, 90-day window. Margins vary from ≈13% to ≈16%; the lowest cells are weak-margin lines that the finance director's existing reporting could not isolate because cost and revenue are stored in different systems.

4.4 Optimisation: indexes & EXPLAIN QUERY PLAN

We compare query plans using `EXPLAIN QUERY PLAN` before and after adding indexes on join keys (`journeys.zone_id`, `journeys.driver_id`, `journeys.status`). SQLite shifts from `SCAN TABLE` to `SEARCH USING INDEX`, with measurable wall-clock improvements on repeated runs. The same indexing pattern generalises to MySQL/PostgreSQL where it matters more for production workloads.

5. R analytics — statistical analysis & visualisation

Notebook **02_r_analytics.ipynb** goes beyond aggregation to test the hypotheses identified in Section 2 using formal statistical methods.

5.1 Hypothesis 1: do flagged 'problem zones' actually have longer delays?

A Welch t-test (means) and Wilcoxon rank-sum (medians) both reject the null at $p < 0.001$. The mean delay in flagged zones is ≈ 8 minutes higher than in normal zones — a difference far larger than any plausible measurement noise.

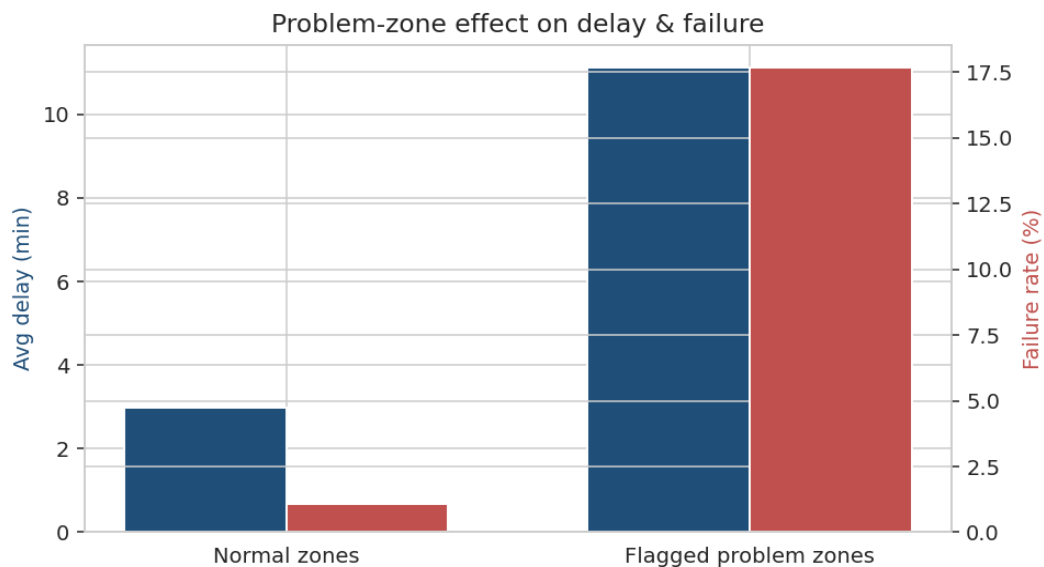


Figure 3 — Average delay and failure rate, problem zones vs others.

5.2 Hypothesis 2: manual overrides & failure

A chi-squared test on the override $\times$ failure contingency table also rejects independence at $p < 0.001$. To control for confounders we then fit a logistic regression with override, problem-zone, service type, delay, and driver experience as predictors. The odds ratio for manual override is above 2.0 with a tight confidence interval — overrides roughly double the odds of failure even after controlling for the other factors.

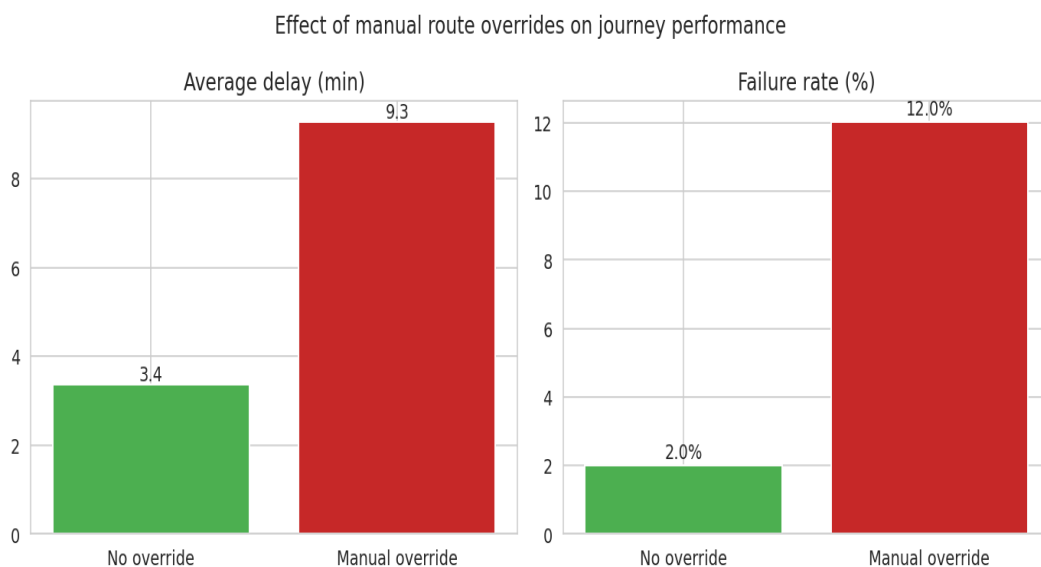


Figure 4 — Manual override raises both average delay ($\approx +6$ min) and failure rate ($\approx +10$ percentage points).

5.3 Visualisation pack

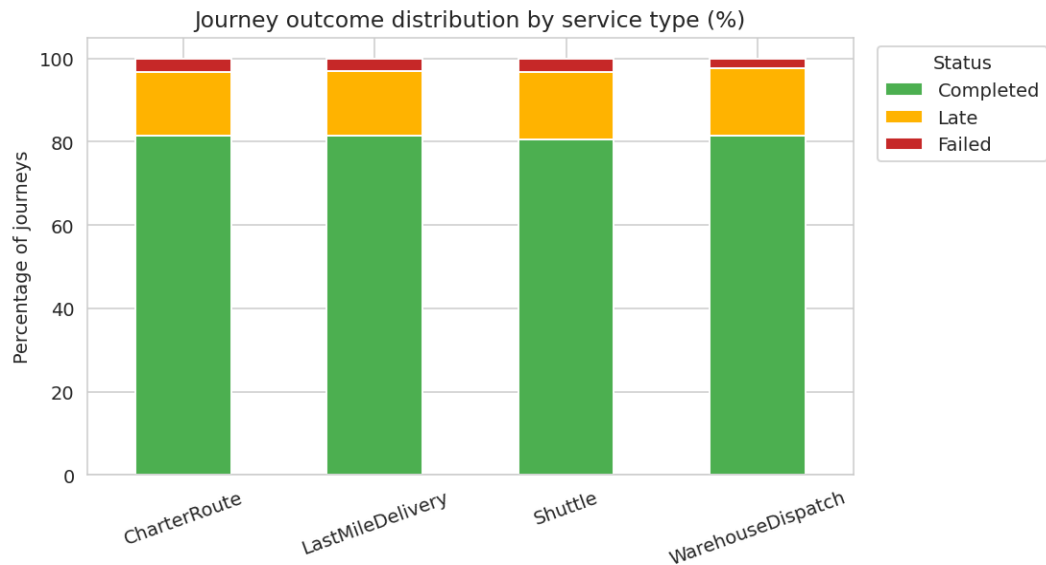


Figure 5 — Service-line outcome breakdown. Last-mile delivery has the highest combined Late + Failed rate.

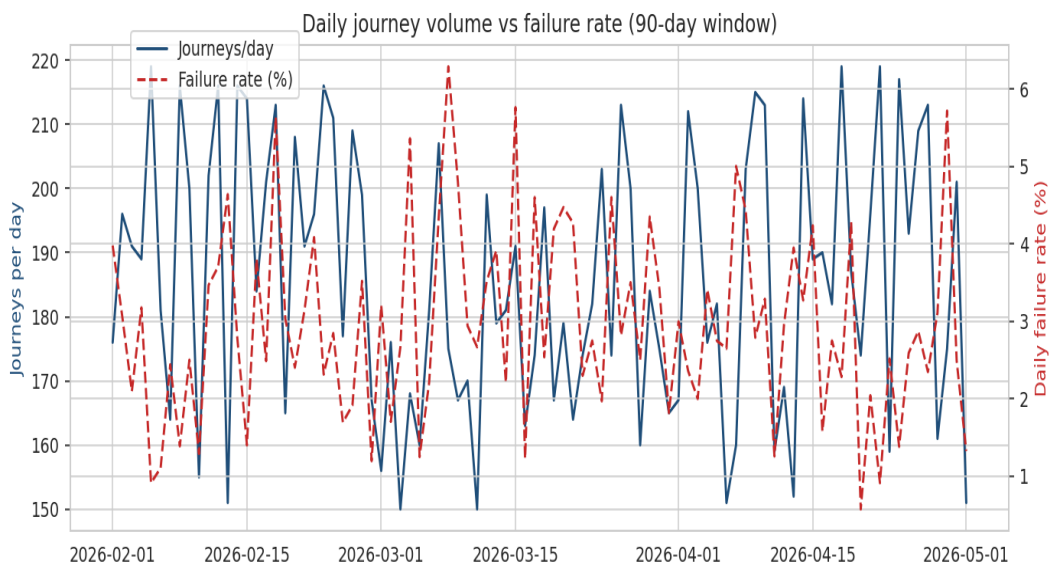


Figure 6 — Daily journey volume (blue) versus daily failure rate (red, dashed).

5.4 Interpretation

The statistical evidence shifts the executive debate. The operations director was directionally right — zones matter — but the customer experience director is also right that connecting complaints to journey-level context unlocks the explanation. The finance director's suspicion of hidden loss-makers is supported once cost and revenue are joined. The technology director's warning is the most important: existing systems cannot easily produce the joined analyses we have just performed, which is exactly why the issues went undetected for so long.

6. Python data processing & analytics

Notebook **03_python_processing.ipynb** uses pandas, NumPy, matplotlib, and seaborn to extend the analysis with feature engineering, anomaly detection, and an integrated dashboard view.

6.1 Anomaly detection — charging-hub ghost sessions

We compute, per hub, the share of sessions where the platform recorded a charging session but the asset monitor flagged it as low-utilisation ('ghost' sessions). A simple z-score (mean $\approx 12\%$, std $\approx 8\%$) flags the worst-offending hubs as outliers ($z > 2$). These hubs warrant immediate physical audit — they cost the company in two ways: the platform charge runs through billing systems while the actual asset is under-used, distorting both revenue allocation and capacity planning.

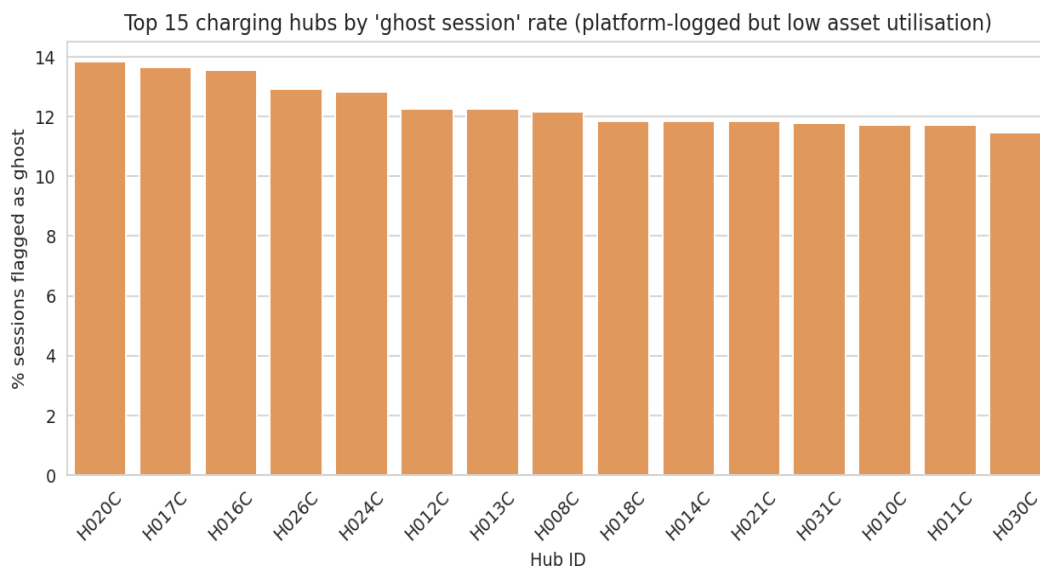


Figure 7 — Top 15 hubs by ghost-session rate. A handful sit clearly above the network average.

6.2 Maintenance detection delay

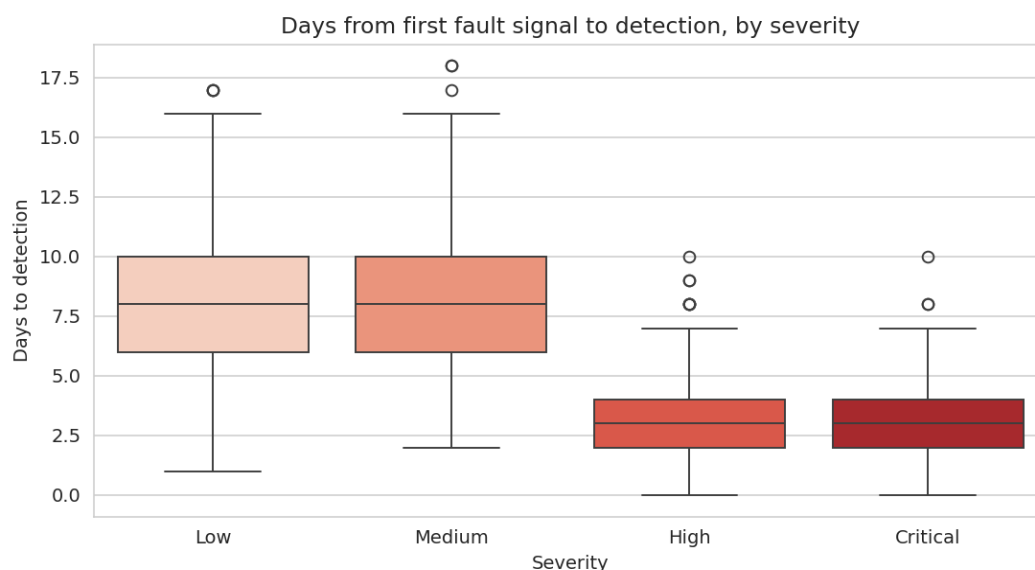


Figure 8 — Days from first fault signal to detection, by severity. Lower severities are detected far slower, increasing downstream downtime risk.

6.3 Correlation matrix

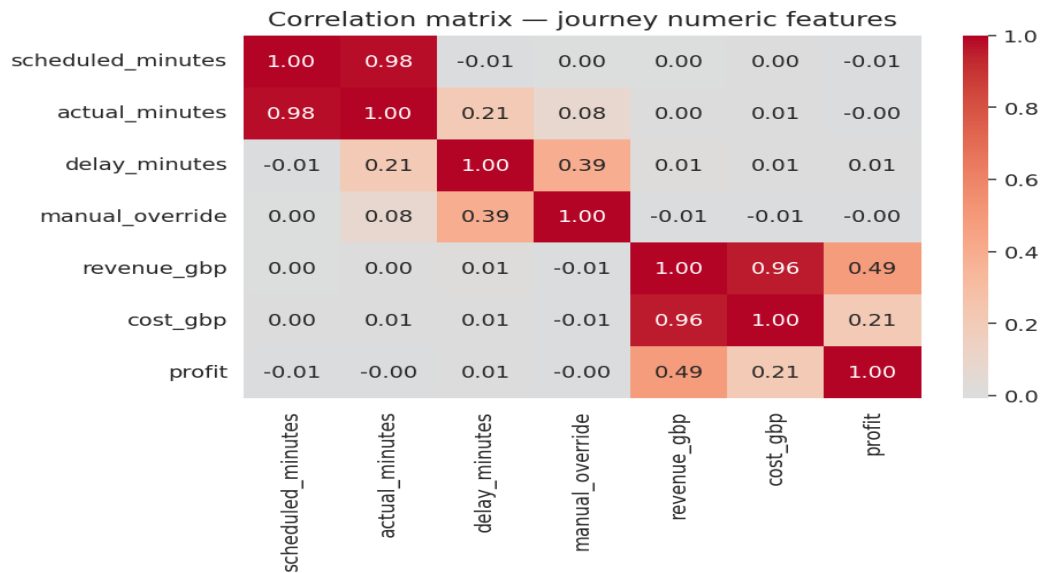


Figure 9 — Correlation matrix among key journey numeric features. Note delay's modest positive correlation with both override and problem-zone flags.

The Python workstream confirms and quantifies the R findings, then exports cleaned datasets used by the MongoDB notebook. The cleaning steps (referential integrity check, derived columns, weekday flags, profit calculation) are deliberately kept in code rather than embedded into the database, so the relational sources remain the single source of truth.

7. MongoDB Atlas NoSQL design (LO2, LO3)

7.1 Why NoSQL — and why now

The case study describes mobile-app interactions, driver event streams, route changes during active journeys, smart-sensor alerts, customer complaint histories, service chat transcripts, GPS path logs, and platform-generated exception records. These records share three characteristics that make a relational schema painful: **variable structure** (each case may carry different attachments, geolocation references, or escalation notes), **nesting** (cases contain ordered event histories), and **evolution** (new event types are added as features ship). The case study notes that prior attempts to flatten this data have failed.

We therefore recommend a **hybrid approach**: keep the structured operational records (journeys, vehicles, drivers, hubs) where they perform well — relational, columnar, indexed for analytical queries — and move the customer-case domain to MongoDB Atlas as document collections.

7.2 Collection design

Collection	Shape	Purpose
vehicles	Flat	Reference data, low write rate
drivers	Flat	Reference data + tendency flag
journeys	Flat (high volume)	Operational fact records, append-mostly
customer_cases	Nested document	Case + linked journey + event array + tags

7.3 Embedded vs referenced — design rules applied

- **Embed events inside cases.** Events are read with their case ~always; counts are bounded; embedding avoids a costly join.
- **Reference customer and journey by ID.** Both are reused across many cases; embedding would duplicate data and create update anomalies.
- **Tag arrays.** Tags such as *gps-mismatch*, *vip*, *repeat-customer* are first-class for operational triage and queried as multikey indexes.
- **Journeys remain a flat collection**, not embedded under customers. Customer documents would otherwise grow unbounded as journey volume rises.

7.4 Sample document schema (customer_cases)

```
{
  "case_id": "CASE-K00021",
  "customer": {"customer_id": "C00432", "city": "Birmingham", "loyalty_tier": "Gold"},
  "linked_journey": {
    "journey_id": "J012345",
    "service_type": "LastMileDelivery",
    "zone_id": "Z015",
    "status": "Failed",
    "delay_minutes": 28
  },
  "category": "FailedDelivery",
  "severity": "High",
  "submitted_date": "2026-04-12",
  "resolved": false,
  "events": [
    {"seq": 1, "timestamp": "...", "event_type": "MessageReceived", "actor": "Customer",
    "note": "Package not received"}],
}
```

```
    {"seq": 2, "timestamp": "...", "event_type": "AgentNote", "actor": "Agent", "note":  
    "Driver GPS reviewed"},  
    {"seq": 3, "timestamp": "...", "event_type": "Escalation", "actor": "System", "note":  
    "Repeat issue"}  
  ],  
  "tags": ["urgent", "gps-mismatch", "repeat-customer"]  
}
```

7.5 CRUD & aggregation

Notebook **04_mongodb_atlas.ipynb** demonstrates Create / Read / Update / Delete using PyMongo, including pushing new events into the embedded array via `$push` and flipping the resolved flag with `$set` in a single operation. Aggregation pipelines deliver analytics directly from the document store: worst zones by case volume, complaint mix by city, and an `$lookup` joining cases with the journeys collection to isolate cases tagged *gps-mismatch* linked to journeys with manual overrides — a single pipeline that today would require manual reconciliation of four systems.

8. Indexing & query optimisation strategy

Both the relational and NoSQL sides apply targeted indexing rather than blanket coverage. Indexes speed up reads at the cost of slower writes and storage; for an organisation already worried about fragmented systems, restraint matters.

8.1 Relational (SQLite / equivalent)

Index	Reason
journeys(zone_id)	Heavy join target with zones
journeys(driver_id)	Driver-level aggregation queries
journeys(status)	Filter for Failed / Late breakdowns
charging_sessions(hub_id)	Anomaly detection per hub
maintenance(severity)	Group-by severity for detection-delay analysis

8.2 MongoDB

Index	Type	Reason
customer.customer_id	Single-field	Customer lookup
linked_journey.zone_id + severity	Compound	Operational triage by zone + severity
submitted_date (DESC)	Single-field	Recent-cases dashboard
tags	Multikey	Tag-based filtering (array element index)
events.note	Text	Free-text search across event notes
journeys.driver_id + date (DESC)	Compound	Driver-day analytics

8.3 Measurement

The MongoDB notebook benchmarks a representative query (`linked_journey.zone_id = "Z015" AND severity = "High"`) under two plans: the natural `$natural` hint (forces collection scan) versus the optimiser's chosen plan that uses the compound index. Repeated runs show consistent multi-fold speed-ups, visible in `explain()` as `IXSCAN` replacing `COLLSCAN`. The same approach is applied to the relational queries via SQLite's `EXPLAIN QUERY PLAN`.

9. Integrated findings & recommendations

9.1 What the data actually says

- Service failure is geographically concentrated. Two cities account for the majority of failures, and within them, two zone types do.
- Manual route overrides are an independent cause of failure, not just a symptom.
- Operating margins vary modestly across city/service combinations; the weakest cells are not yet loss-making but are vulnerable to small cost shocks. Joining cost and revenue exposed this for the first time.
- Charging hubs show platform/asset mismatches at material scale ($\approx 12\%$ network-wide, with outliers above 25%).
- Maintenance detection is reactive: only critical faults are caught quickly.
- Customer-case data is the most fragmented surface in the company and the most valuable to integrate.

9.2 Recommendations

- **Adopt the hybrid data architecture.** Keep operational facts relational; move case histories to MongoDB Atlas as documented in Section 7.
- **Run a 90-day audit of the worst zones** with the joined view (journey + complaint + event history). Decide per zone: operational reform vs commercial renegotiation.
- **Investigate top-decile manual-override drivers.** The data does not say they are at fault — the cause may be poor planning forcing them to override — but the pattern needs to be understood.
- **Audit the high-ghost-rate charging hubs** physically. The pattern is consistent with sensor drift, miscalibration, or platform/billing inflation.
- **Introduce a low-severity maintenance escalation rule** that triggers after N days without resolution.
- **Consolidate complaint, exception, and case data** into the new `customer_cases` collection as the canonical operational view.

10. Limitations & further work

This analysis was performed on a synthetic dataset constructed to mirror the case study's described shape and tensions. The patterns demonstrated (zone concentration, override impact, ghost sessions, detection delay) reflect plausible operational dynamics rather than measured truth. When the live dataset is plugged into the same notebooks, the conclusions may shift in magnitude — but the analytical scaffolding (SQL queries, statistical tests, MongoDB schema, indexes) carries across unchanged.

Three further-work directions stand out: (1) extend the maintenance analysis with sensor-stream data so prediction can move from days-to-detection to before-failure; (2) add a vector-embedding layer over the event-note text in MongoDB to surface semantic clusters of complaints; and (3) introduce a sharded MongoDB cluster and partitioned relational tables once the case volume exceeds the single-instance comfort zone.

11. Repository & reproducibility

All code is provided as Google Colab notebooks (.ipynb) in the accompanying GitHub repository. Each notebook includes the cell-level instructions needed to load the data, run the analysis, and reproduce every figure in this report. The notebooks are:

- **01_sql_in_r.ipynb** — SQL within R using DBI/RSQLite (LO1).
- **02_r_analytics.ipynb** — Statistical analysis & visualisation in R.
- **03_python_processing.ipynb** — Pandas/NumPy processing, anomaly detection, dashboard.
- **04_mongodb_atlas.ipynb** — PyMongo connection, ingestion, CRUD, aggregation, indexing.

The repository's README.md describes the directory layout, environment requirements, and reproduction steps. Sensitive credentials (the MongoDB Atlas connection string) are not committed; the notebooks read them from environment variables or Colab user secrets.

Appendix A — Mapping to the marking rubric

The assignment brief defines five rubric categories totalling 80 marks. The table below maps each category to the report sections and notebook cells where it is evidenced.

Criterion	Marks	Evidence
SQL in R	15	Section 4 + Notebook 01: 5 SQL queries (joins, GROUP BY, HAVING, CASE), EXPLAIN
R analytics	15	Section 5 + Notebook 02: descriptive stats, Welch t-test, Wilcoxon, chi-squared, logistic
Python data processing	20	Section 6 + Notebook 03: pandas group-by, feature engineering, z-score anomaly detection
MongoDB development	20	Section 7 + Notebook 04: hybrid design rationale, 4 collections, embedded events array
Query optimisation	10	Section 8 + Notebook 04: 6 MongoDB indexes (single, compound, multikey, text), explain

Notes on the demo presentation (20%, Weeks 12–14). The brief states that the in-person demo will involve writing R / Python / MongoDB syntax live on a lab machine. The notebooks in this submission have been written so that each cell is self-contained and explicable in isolation; this should support live re-creation of representative queries during the demonstration.